INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, PUNE

Department of Computer Science and Engineering

ArogyaMitra: A Secure, Explainable, Fail-Closed AI-Assisted Primary-Healthcare Decision Support System

A Project Report Submitted in Partial Fulfillment of the Requirements
for the Degree of Bachelor of Technology in Computer Science and Engineering

Submitted By:
Satyam Keshari
MIS: 112315168
B.Tech CSE (Semester VII)
Dept. of Computer Science & Engineering

Under the Guidance of:
[CONFIRM TITLE] Ashish Singh
Adjunct Assistant Professor
Dept. of Electronics & Communication Engineering
IIIT Pune

Department of Computer Science and Engineering
Indian Institute of Information Technology, Pune
November 2026 (Academic Year 2026–2027)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, PUNE

Department of Computer Science and Engineering

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "ArogyaMitra: A Secure, Explainable, Fail-Closed AI-Assisted Primary-Healthcare Decision Support System" submitted by Satyam Keshari (MIS: 112315168) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at the Indian Institute of Information Technology, Pune, is a bonafide record of the work carried out under my supervision during the academic year 2026–2027.

The matter embodied in this report has not been submitted to any other University or Institute for the award of any degree or diploma. The system was designed and benchmarked as an academic decision support prototype in accordance with institutional guidelines.

_____	_____	_____
[CONFIRM TITLE] Ashish Singh	[CONFIRM HOD NAME]	External Examiner
Project Supervisor	Head of Department	Evaluation Committee
Adjunct Assistant Professor, ECE	Dept. of CSE	IIIT Pune
IIIT Pune	IIIT Pune	

DECLARATION & PLAGIARISM UNDERTAKING

I, Satyam Keshari (MIS: 112315168), student of Bachelor of Technology in Computer Science and Engineering, Semester VII, IIIT Pune, hereby declare that the work presented in this project report entitled "ArogyaMitra: A Secure, Explainable, Fail-Closed AI-Assisted Primary-Healthcare Decision Support System" is my original engineering implementation.

I affirm that: (1) All components, pipelines, container definitions, and test suites were engineered directly within the repository; (2) All reported empirical metrics (cross-validation accuracies, ECE, throughput, test counts, vulnerability scans) correspond to verifiable logs in docs/evidence/ and are cited with file:line; (3) Generative AI tools (LLM assistants) were used for code refactoring, test script scaffolding, and document formatting; all architectural decisions, mathematical derivations, bug discoveries, and empirical verifications were performed directly by the student.

CONFLICT OF INTEREST STATEMENT

The author declares that there are no financial, commercial, or personal conflicts of interest. The ArogyaMitra system was engineered strictly as an academic research and engineering prototype in clinical decision support. No commercial sponsorship was accepted.

The machine learning models were trained and benchmarked exclusively on synthetic clinical symptom datasets (304 unique rows). The system has not been tested on live human patients, has not accessed identifiable protected health information, and is not certified as a medical device by CDSCO or the FDA.

_____

Satyam Keshari (MIS: 112315168)
Dept. of CSE, IIIT Pune | Date: November 01, 2026

ACKNOWLEDGEMENTS

I express my sincere gratitude to my project supervisor, [CONFIRM TITLE] Ashish Singh, Adjunct Assistant Professor, Department of Electronics and Communication Engineering, IIIT Pune, for his guidance, periodic review, and valuable feedback throughout the course of this project. His reviews helped ensure that the engineering milestones and project goals remained on track.

I extend my appreciation to the Head of Department, [CONFIRM HOD NAME], Department of Computer Science and Engineering, and [CONFIRM DIRECTOR NAME], Director, IIIT Pune, for providing the institutional computing facilities and academic environment essential for completing this B.Tech capstone project.

I also thank the faculty members of the Department of Computer Science and Engineering for their evaluation and questions during project reviews, which helped refine the system requirements and testing approach.

Finally, I thank my family and peers for their continuous support and patience throughout the duration of this work.

Satyam Keshari
MIS: 112315168

ABSTRACT

In primary healthcare settings across rural and semi-urban India, community health workers such as Accredited Social Health Activists (ASHAs) conduct patient intake under limited diagnostic resources and high patient loads. Digital symptom checkers have been proposed to assist triage, but systematic reviews (notably Wallace et al. [1]) show that digital symptom checkers exhibit low primary diagnostic accuracy (19–37.9%) and variable triage accuracy (48.8–90.1%), with an alignment of errors that can create patient safety hazards. Furthermore, theoretical machine learning literature (Niculescu-Mizil & Caruana [2]) demonstrates that Naive Bayes classifiers push predicted probabilities toward 0 and 1 due to conditional feature independence assumptions, producing uncalibrated estimates.

To address these operational risks, this project implements ArogyaMitra, a decision support prototype designed for frontline health workers. The system incorporates: (1) a stateless Bernoulli Naive Bayes classifier trained across 41 disease classes and 131 symptoms, accompanied by closed-form additive log-odds feature attributions; (2) a deterministic red-flag screening layer that overrides model predictions when emergency symptoms are present; (3) a guarded Large Language Model narrative layer with ordered model cascade, a 10-second per-attempt timeout, a 20-second hard cap, and whole-phrase confidence filtering backed by pre-compiled fallback templates; and (4) a containerized architecture with TLS 1.2/1.3, Argon2id password hashing, field-level AES-256-GCM encryption, and optimistic concurrency controls.

Empirical evaluation on a deduplicated benchmark of 304 unique rows (derived by removing 4,616 exact duplicate rows from a 4,920-row dataset) using 5x3 repeated stratified cross-validation showed BernoulliNB achieving Top-1 accuracy of 1.000 ± 0.000 at 0% dropout, 0.989 ± 0.014 at 30% dropout, and 0.919 ± 0.023 at 50% dropout, compared to Tuned LightGBM (0.989, 0.864, 0.683) and Tuned XGBoost (0.984, 0.868, 0.720). Load testing across 2,400 requests in test mode showed system throughput reaching 14.7 req/s under 50 concurrent users with 100% HTTP 200 responses, while server-side pagination and projection reduced worker list p50 latency from 193.0 ms to 76.2 ms. Automated test suites verify 321 prototype test cases, 20 golden cases, 11 Playwright journeys, and zero fixable Critical/High (Trivy --ignore-unfixed) container vulnerabilities.

Keywords: *Clinical Decision Support, Primary Healthcare Triage, Bernoulli Naive Bayes, Uncalibrated Posterior Estimation, Fail-Closed Architecture, Large Language Models, Container Security.*

TABLE OF CONTENTS

Bonafide Certificate . . . . . **ii**
Declaration & Conflict of Interest . . . . . **iii**
Acknowledgements . . . . . **iv**
Abstract . . . . . **v**
List of Figures & List of Tables . . . . . **vi**
List of Abbreviations . . . . . **vii**
Chapter 1: Introduction & Literature Review **1**
1.1 Motivation & Primary Healthcare Challenges in Rural India 1
1.2 Comprehensive Literature Review 2
1.2.1 Review of Symptom Checkers, Decision Support, and Calibration . . 2
1.2.2 Comparison with Existing Work 3
1.3 Problem Statement & Design Objectives 4
1.4 Design Evolution 4
1.5 Report Outline & Organization 5
Chapter 2: System Requirements & Threat Modeling **6**
2.1 Functional Requirements (FR-1 to FR-10) 6
2.2 Non-Functional Requirements (NFR-1 to NFR-8) 6
2.3 Threat Model & Security Analysis (STRIDE Matrix) 7
2.4 Clinical Safety & Ethics Requirements 7
Chapter 3: System Architecture & Design **9**
3.1 Architectural Overview & Container Topology 9
3.2 Machine Learning Inference & Calibration Engine 10
3.2.1 Bernoulli Naive Bayes Mathematical Formulation 10
3.2.2 Feature Engineering & Vocabulary Alignment 11
3.2.3 Explainability via Top-Feature Attribution 11
3.2.4 Calibration Theoretical Background & Analysis 11
3.2.5 Comparative Model Selection 12
3.3 Rule-Based Clinical Red-Flag Override System 12
3.4 Multi-Tier LLM Explanation & Resilient Fallback Layer 13
3.5 End-to-End Clinical Consultation Workflow 14
3.6 Security Architecture & Cryptographic Implementation 14
3.7 Database Design, Concurrency & Data Integrity 15
3.8 Logging and Audit System 15
Chapter 4: Empirical Evaluation & Experimental Results **16**
4.1 Dataset Profile & Methodological Integrity 16
4.2 Machine Learning Performance & Robustness 16
4.3 Load Testing, Concurrency & System Throughput 17
4.4 Automated Testing & Verification Suite 19
4.5 Security Vulnerability & Dependency Auditing 20
4.6 Literature-Linked Discussion & Analysis 21
4.7 Limitations & Threats to Validity 21
Chapter 5: Conclusion & Future Scope **23**
5.1 Project Summary & Achievements 23
5.2 Lessons Learned & Engineering Insights 23
5.3 Future Work & Roadmap 23
References **25**
Appendix A: Engineering Challenges, Root Causes, and Architectural Fixes . . . **26**
Appendix B: Reproduction Commands Playbook **28**

LIST OF FIGURES

Figure 3.1: Container Topology, Network Boundaries, and Data Tiers **9**
Figure 3.2: Consultation Lifecycle State Machine and Decision Gates **14**
Figure 3.3: Hierarchical Request Timeout Budgets and Guardrail Cascade . . . **14**
Figure 4.1: Top-1 Accuracy and Expected Calibration Error (ECE) vs. Dropout . . **17**
Figure 4.2: Dataset Provenance: Raw 4,920 Rows vs. 304 Deduplicated Unique Rows . . **16**
Figure 4.3: Gateway Throughput and Percentile Latencies Across Concurrency . . **18**
Figure 4.4: Worker List Latency Before vs. After Pagination and Projection . . **18**
Figure 4.5: User Interface Journeys: Worker Intake, Red-Flag Block, Pagination, Outage . . **19**

LIST OF TABLES

Table 1.1: Comprehensive Literature Review Summary Matrix **2**
Table 1.2: Systematic Comparison: Clinical Concerns vs. ArogyaMitra Implementations . . **3**
Table 2.1: System Functional Requirements Specification (FR-1 to FR-10) . . **6**
Table 2.2: System Non-Functional Requirements: Targets vs. Measured Results . . **6**
Table 2.3: STRIDE Threat Model and Architectural Security Controls **7**
Table 3.1: Emergency Red-Flag Trigger Symptoms (config/red_flags.yaml) . . . **12**
Table 3.2: Container Technology Stack and Component Allocation **9**
Table 4.1: Repeated Stratified 5x3 Cross-Validation Accuracy Across Dropout . . **17**
Table 4.2: Expected Calibration Error (ECE) Across Dropout Rates **17**
Table 4.3: Load Test Benchmark Results Across Concurrency Levels (2,400 Requests) . . **18**
Table 4.4: Worker List Latency Before vs. After Pagination and Projection . . **18**
Table 4.5: Automated Test Suite Summary from Actual Test Runners (321 Tests) . . **19**
Table 4.6: OWASP ZAP Dynamic Security Baseline Scan Dispositions (6 Alerts) . . **20**
Table A.1: Engineering Challenges, Root Causes, and Architectural Remediation Fixes . . **26**

LIST OF ABBREVIATIONS

ABDM — Ayushman Bharat Digital Mission
AES-GCM — Advanced Encryption Standard in Galois/Counter Mode
ANM — Auxiliary Nurse Midwife
ASHA — Accredited Social Health Activist
CDSCO — Central Drugs Standard Control Organisation (India)
CSP — Content Security Policy
CV — Cross-Validation
DPDP — Digital Personal Data Protection Act, 2023 (India)
ECE — Expected Calibration Error
FHIR — Fast Healthcare Interoperability Resources
HSTS — HTTP Strict Transport Security
HWC — Health and Wellness Centre
JWT — JSON Web Token
LLM — Large Language Model
MAP — Maximum A Posteriori
ML — Machine Learning
NB — Naive Bayes
NFR — Non-Functional Requirement
OCC — Optimistic Concurrency Control
PHC — Primary Health Centre
PII — Personally Identifiable Information
QUADAS-2 — Quality Assessment of Diagnostic Accuracy Studies 2
RBAC — Role-Based Access Control
STRIDE — Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
TLS — Transport Layer Security
UI — User Interface
ZAP — Zed Attack Proxy (OWASP)

Chapter 1: Introduction & Literature Review

1.1 Motivation & Primary Healthcare Challenges in Rural India

In developing healthcare systems with wide geographic dispersion, delivering consistent primary healthcare remains a fundamental public health challenge. In India, the primary healthcare pyramid relies heavily on community-level frontline workers, primarily Accredited Social Health Activists (ASHAs) and Auxiliary Nurse Midwives (ANMs) stationed across Health and Wellness Centres (HWCs) and Sub-Centres. These frontline workers operate under severe operational constraints: high patient-to-physician ratios that frequently exceed 1:10,000 in rural districts, rudimentary diagnostic infrastructure, and significant travel distances separating peripheral villages from referral hospitals.

When rural patients present with acute symptoms, the frontline health worker must make high-stakes triage decisions: determining whether an individual can be safely managed with supportive care, requires scheduled primary health centre (PHC) physician evaluation, or demands immediate emergency transfer to a district hospital. Errors in this triage process carry severe clinical consequences. Under-triage of acute conditions—such as myocardial infarction presenting as epigastric discomfort—leads to preventable mortality. Conversely, systematic over-triage congests secondary and tertiary facilities, imposing travel and consultation costs on rural households.

While digital health initiatives under the Ayushman Bharat Digital Mission (ABDM) have expanded electronic health record adoption, decision support tools for community health workers remain limited. Commercial symptom checkers and diagnostic mobile applications often fail in this operating environment. They are typically engineered as consumer tools that require continuous broadband connectivity, provide no interpretable clinical rationales, and lack fail-closed safety designs. When an underlying algorithmic service experiences network timeout or server failure, such systems frequently crash or return empty screens, leaving frontline workers without guidance.

In this project, I address these challenges by engineering ArogyaMitra, a secure, explainable, and fail-closed AI-assisted clinical decision support system prototype tailored for community healthcare workers. The system does not attempt to replace physician diagnosis. Instead, it serves as an assistive screening aid that provides differential disease estimation, enforces deterministic safety overrides for emergencies, operates with bounded latency, and protects patient privacy through containerized security.

[Primer for Evaluation: Naive Bayes, Probabilistic Calibration, and Clinical Triage] *1. Bernoulli Naive Bayes Classifier: Treats each presenting symptom as a binary indicator variable (present or absent) across 131 clinical features and 41 disease classes. By assuming conditional independence among symptoms given the disease, the model computes log-posterior class scores by summing log-likelihood ratios. This avoids matrix inversions and yields inference times of approximately 1.8 ms on commodity CPUs.*

2. Probabilistic Calibration: In statistical learning, a classifier is well-calibrated if a predicted score of 0.8 corresponds to an empirical 80% event frequency. Naive Bayes violates this condition because multiplying correlated symptom likelihoods artificially inflates the posterior odds, pushing scores toward 0 and 1. Post-hoc calibration methods (such as Platt scaling) can overfit on scarce clinical datasets; in my experiments on 304 unique rows, Platt scaling worsened log loss from 0.0061 to 0.1643. Consequently, ArogyaMitra explicitly displays scores as uncalibrated estimates.

3. Deterministic Red-Flag Triage Layer: Because statistical classifiers can err when presented with rare or atypical symptom combinations, ArogyaMitra decouples emergency detection from machine learning. A deterministic rule engine scans for life-threatening symptoms (e.g., chest pain, coma, breathlessness) before the model runs. If triggered, it immediately locks the triage output to EMERGENCY, preventing model misclassification from causing clinical under-

triage.

1.2 Comprehensive Literature Review

1.2.1 Review of Symptom Checkers, Decision Support, and Calibration

The academic literature evaluating algorithmic decision support and symptom checking tools reveals important discrepancies between theoretical predictive accuracy and clinical safety. A systematic review by Wallace et al. (2022) [1] analyzed 10 studies evaluating 48 digital symptom checkers using the Quality Assessment of Diagnostic Accuracy Studies (QUADAS-2) methodology. Their key findings include:

1. Diagnostic Accuracy: Across evaluated platforms, primary diagnostic accuracy was low, ranging between 19.0% and 37.9%. Even when evaluating top-3 or top-5 differential accuracy, symptom checkers showed limited precision compared to practicing general practitioners.

2. Triage Accuracy: Triage decision accuracy (determining urgency) ranged from 48.8% to 90.1%, but exhibited wide variation across different commercial checkers evaluating identical clinical vignettes.

3. Safety Hazards and Swiss-Cheese Failure Model: Wallace et al. noted that symptom checkers present patient safety hazards when errors align across stages—such as an incorrect primary diagnosis cascading into an inappropriately delayed triage recommendation.

4. Need for Independent Evaluation: The authors concluded that digital symptom checkers require independent clinical validation on real-world data and formal medical device regulation before deployment in public healthcare.

Complementing clinical triage evaluations, the theoretical behavior of probabilistic classifier calibration was investigated by Niculescu-Mizil & Caruana (2005) [2] across 10 supervised learning families. They showed that:

1. Distortion in Naive Bayes Estimators: Naive Bayes pushes predicted probabilities toward 0 and 1 because its conditional independence assumption multiplies marginal likelihoods, causing redundant evidence to inflate posterior log-odds.

2. Data Scarcity and Isotonic Regression: The authors demonstrated that non-parametric calibration (Isotonic regression) requires substantial validation data and performs worse than parametric Platt scaling when data is scarce.

Table 1.1 summarizes six seminal contributions across clinical symptom checking, Bayesian classification, interpretability, and natural language generation that directly informed the design of ArogyaMitra.

Table 1.1: Comprehensive Literature Review Summary Matrix

Author & Ref	Venue	Methodological Summary	Relevance to ArogyaMitra
Wallace et al. (2022) [1]	*npj Digital Medicine*	Systematic review of 10 studies evaluating digital symptom checkers using QUADAS-2.	Directly motivates positioning ArogyaMitra as decision support rather than diagnosis, adding deterministic red flags, and fail-closed design.
Niculescu-Mizil & Caruana (2005) [2]	*Proc. ICML*	Empirical study of probability calibration across 10 supervised learning algorithms.	Explains why BernoulliNB outputs extreme posteriors; justifies labeling scores as uncalibrated estimates. Our Platt scaling trial

			was an independent experiment.
Domingos & Pazzani (1997) [3]	*Machine Learning*	Theoretical and empirical analysis of Naive Bayes optimality under zero-one loss.	Justifies deploying Bernoulli Naive Bayes for discrete symptom triage despite correlated clinical presentations.
Ke et al. (2017) [4]	*NeurIPS*	Introduced LightGBM with Gradient-based One-Side Sampling and Exclusive Feature Bundling.	Served as the comparative tree ensemble in our nested cross-validation benchmark on the 304 unique rows.
Lundberg & Lee (2017) [5]	*NeurIPS (SHAP)*	Unified framework for interpreting model predictions using game-theoretic Shapley values.	Underpins our mathematical derivation of exact additive log-odds, ensuring symptom attributions sum to class log-evidence without sampling.
Ji et al. (2023) [6]	*ACM Computing Surveys*	Comprehensive survey of hallucination in Large Language Models.	Motivated our architectural restriction: the LLM never classifies diseases and only formats explanations, backed by deterministic fallback templates.

1.2.2 Comparison with Existing Work

To clarify the technical positioning and safety boundaries of ArogyaMitra, Table 1.2 contrasts specific clinical concerns documented in literature against conventional symptom checkers and the verifiable implementations within this repository.

Table 1.2: Systematic Comparison: Clinical Concerns vs. ArogyaMitra Implementations

Clinical / Technical Concern	Literature Citation	ArogyaMitra Implementation	Unvalidated Boundary
Low and variable diagnostic accuracy across symptom checkers (primary diagnosis 19–37.9%)	*Wallace et al. (2022) [1]*	Evaluated top-3 differential diagnoses and dropout robustness across 0%, 30%, and 50% missing symptoms using 5x3 repeated stratified CV.	Evaluated on synthetic data (304 unique rows); zero real-patient cohort validation.
Triage and patient safety hazards from incorrect diagnosis alignment (Swiss-cheese failure)	*Wallace et al. (2022) [1]*	Implemented two-tier triage safety: Decision A red-flag emergency screening, low-confidence escalation (<70%), and fail-closed HTTP 503 outage behavior.	Red-flag rules remain in DRAFT status pending formal institutional clinical review.
Need for real-world studies, clinical trial evidence, and medical device regulation	*Wallace et al. (2022) [1]*	Explicitly scoped as an uncertified educational decision-support prototype; added DPDP-aware audit logging and data protection controls.	Not certified under CDSCO or MDR 2017; has not undergone prospective clinical trials.
Naive Bayes pushes predicted probabilities toward 0 and 1 due to independence assumptions	*Niculescu-Mizil & Caruana (2005) [2]*	Labeled confidence scores as 'estimate, not a clinical probability' in UI; empirical thresholds tuned on held-out data.	Confidence scores still saturate near 100% when multiple correlated symptoms match.
Calibration methods require separate calibration data and may degrade on small datasets	*Niculescu-Mizil & Caruana (2005) [2]*	Empirically tested Platt scaling on 304 unique rows; verified log loss worsened from 0.0061 to 0.1643; deployed uncalibrated raw posterior estimates.	No large real-world clinical dataset available to train a reliable non-parametric calibration map.

Feature attribution instability and stochastic sampling variance in local surrogates (LIME)	*Lundberg & Lee (2017) [5]*	Derived exact additive log-odds evidence weights directly from Bernoulli likelihoods, satisfying additivity without sampling.	Feature attributions reflect statistical model log-odds, not causal biological pathophysiology.
Hallucination of diagnoses, dosages, and medical facts in Large Language Models	*Ji et al. (2023) [6]*	Architectural boundary isolation: LLM never predicts the disease class; it only formats pre-computed evidence; fallback templates with zero drug dosages.	Template library covers only the 41 trained conditions; requires ongoing physician curation.

1.3 Problem Statement & Design Objectives

Problem Statement: Community health workers in primary healthcare settings require an auditable, privacy-preserving, and fail-closed decision support tool to triage acute presentations without relying on black-box predictions, ungrounded generative text, or unvetted cloud infrastructure.

To address this problem, I engineered ArogyaMitra according to five design objectives:

1. Objective 1 — Probabilistic Explainability: Deploy an inference engine where every differential disease score is mathematically traceable to specific symptom attributions, avoiding black-box models.

2. Objective 2 — Deterministic Red-Flag Override: Ensure that clinical red-flag symptoms (e.g., chest pain, acute dyspnea, coma) immediately trigger emergency triage, superseding statistical model predictions.

3. Objective 3 — Guarded LLM Narrative Generation: Confine the Large Language Model strictly to formatting summaries, enforcing a 10s per-attempt timeout and a 20s overall cap backed by deterministic medical templates.

4. Objective 4 — Fail-Closed System Behavior: In the event of microservice failure or database unreachability, the system must not output mock predictions; it must reject unsafe operations and return explicit HTTP 503 error states.

5. Objective 5 — Layered Data Security: Implement cryptographic protections across trust boundaries, including field-level AES-256-GCM encryption, Argon2id credential hashing, and container network isolation.

1.4 Design Evolution

The architecture of ArogyaMitra evolved through several engineering iterations as testing revealed limitations in earlier designs. In early versions, the system was structured as a conventional web prototype: a monolithic script exposing a tree-based classifier, an unconstrained generative LLM prompt, and default web application security. Through systematic auditing and benchmarking, I identified several vulnerabilities and made the following design decisions:

1. Rejection of Spurious Model Calibration Claims: When initial tests attempted post-hoc Platt scaling on the dataset, empirical evaluation revealed that fitting a Platt sigmoid on the 304 unique rows degraded log loss from 0.0061 to 0.1643. I determined that fitting parametric sigmoids on scarce class samples introduces boundary distortion. I removed claims of calibrated probabilities and updated the user interface to explicitly state that scores represent uncalibrated posterior estimates from Bernoulli Naive Bayes.

2. Transition from Mock Fallbacks to Fail-Closed Design: Early prototypes included client-side mock fallbacks that returned simulated predictions when backend services were offline. Recognizing that returning canned predictions during an outage creates patient safety hazards, I redesigned the system to be fail-closed: removing all client mocks and ensuring that service outages return an explicit HTTP 503 MODEL_UNAVAILABLE error with clear instructions to seek emergency care.

3. Derivation of Exact Log-Odds Over Sampling Surrogates: Initial experiments with local perturbation surrogates (such as LIME) showed sampling variance and high latency. To achieve fast, deterministic explanations, I derived closed-form Bayesian log-odds attributions directly from the Bernoulli parameter matrix, achieving exact additive feature attributions without sampling overhead.

4. Container Decoupling and Secret Hardening: Static analysis detected fallback encryption keys embedded in source files. I refactored the system into isolated Docker microservices connected over a private bridge network, added environment variable enforcement, and wrote an automated production-guard test verifying that missing encryption keys halt execution immediately.

1.5 Report Outline & Organization

The remainder of this report is organized as follows: Chapter 2 specifies the functional, non-functional, and STRIDE threat modeling requirements. Chapter 3 provides the system architecture, mathematical formulation of Bernoulli Naive Bayes, red-flag screening rules, guarded LLM timeout cascades, cryptographic implementations, and concurrency controls. Chapter 4 presents empirical evaluations, including nested cross-validation across symptom dropout levels, load testing, automated test suite verification, and literature-linked discussion. Chapter 5 concludes the report and discusses future work. Engineering challenges and CLI reproduction playbooks are provided in Appendices A and B.

Chapter 2: System Requirements & Threat Modeling

2.1 Functional Requirements

ArogyaMitra governs clinical triage interactions across frontline community health workers, physicians, and administrative auditors. Table 2.1 specifies the ten functional requirements (FR-1 through FR-10) implemented across the microservice stack.

Table 2.1: System Functional Requirements Specification (FR-1 to FR-10)

Req ID	Requirement Title	Technical Scope
FR-1	**Consultation Intake**	Capture basic demographics and chief complaints, validating inputs against Pydantic schemas.
FR-2	**Vocabulary Mapping**	Convert symptom selections from a 131-symptom vocabulary into binary indicator vectors.
FR-3	**Red-Flag Screening**	Scan symptom inputs against 7 emergency symptoms; immediately assign Emergency triage on match.
FR-4	**Bayesian Inference**	Compute uncalibrated posterior log-odds across 41 disease classes and return top-3 ranked differentials.
FR-5	**Feature Attribution**	Calculate exact positive and negative log-likelihood contributions for each presenting symptom.
FR-6	**Guarded LLM Summary**	Generate clinical summaries via Gemini API subject to strict confidence-word filtering.
FR-7	**Template Fallback**	Engage deterministic pre-compiled templates on LLM timeout (10s/20s) or 429 quota exhaustion.
FR-8	**Worker Decision Gate**	Provide an explicit Accept / Escalate decision interface for health worker review.
FR-9	**Consultation Pagination**	Support indexed server-side pagination (limit, page) and lean projection on GET /consultation/my.
FR-10	**DPDP-aware Retention**	Support patient consent recording, consent withdrawal, and administrator-approved anonymization.

2.2 Non-Functional Requirements: Targets vs. Measured Results

Table 2.2 defines eight non-functional requirements presented as engineering targets alongside their measured evaluation outcomes.

Table 2.2: System Non-Functional Requirements: Targets vs. Measured Results

Req ID	Dimension	Target Specification	Measured Outcome & Pass/Fail Status
NFR-1	**Gateway Latency**	<= 1,500 ms at p95 under load	PASS at C=1 (190.6 ms); FAIL at C=10 (1952.3 ms) and C=50 (5862.8 ms) under local virtualized load.
NFR-2	**ML Micro-Latency**	<= 50 ms inference per query	PASS. Measured 1.8 ms in internal container

			benchmark via vectorized NumPy log-odds.
NFR-3	**Fail-Closed Outage**	Return HTTP 503 on service down	PASS. Verified by test suite (test_phase0.py); rejects requests when ML container is offline.
NFR-4	**Data Encryption**	AES-256-GCM field encryption	PASS. Verified by automated tests; unique 96-bit IV per record; process exits if key is missing.
NFR-5	**Password Security**	Argon2id credential hashing	PASS. Enforced in auth service (3 iterations, 64 MB RAM, 4 parallelism threads).
NFR-6	**State Concurrency**	Prevent lost concurrent writes	PASS. Enforced via Mongoose version key (__v) and conditional atomic updates.
NFR-7	**Transport Security**	TLS 1.2 / TLS 1.3 encryption	PASS. Nginx enforces ssl_protocols TLSv1.2 TLSv1.3 with HSTS and nosniff headers.
NFR-8	**Container Isolation**	Isolated Docker private bridge	PASS. Services run on private bridge arogya-net; MongoDB exposes no public host port.

2.3 Threat Model & Security Analysis (STRIDE Matrix)

To identify potential attack surfaces, I conducted a STRIDE threat modeling analysis across system trust boundaries. Table 2.3 summarizes the identified threat scenarios and implemented countermeasures.

Table 2.3: STRIDE Threat Model and Architectural Security Controls

STRIDE Category	Identified Threat Scenario	Architectural Countermeasure
Spoofing	Adversary forges credentials or intercepts session tokens to submit false triage data.	Short-lived signed JWTs (15-min expiry) with HMAC-SHA256 and Argon2id hashing.
Tampering	Adversary modifies triage urgency scores or alters symptom vectors in transit.	Enforced TLS 1.2/1.3 encryption across external gateways; internal Docker network isolation.
Repudiation	Health worker denies recording a triage decision or clinician denies failing to review.	Structured MongoDB audit logs recording timestamps, user IDs, roles, and client IPs.
Info Disclosure	Eavesdropper accesses patient PII or clinical records through database dumps.	Field-level AES-256-GCM encryption for patient PII; PII scrubbing before LLM input.
Denial of Service	Adversary floods triage endpoints with requests, exhausting CPU or Gemini quota.	Rate limiting (100 req/15 min on API routes); 20s LLM hard cap with template fallback.
Elevation of Priv	Frontline worker accesses administrative routes or modifies physician records.	Role-Based Access Control middleware enforcing role checks ('worker', 'doctor', 'admin').

2.4 Clinical Safety & Ethics Requirements

In accordance with medical software safety guidelines, ArogyaMitra enforces three clinical safety principles:

1. Human-in-the-Loop Operation: ArogyaMitra is architected solely as decision support, not an autonomous diagnostic agent. It cannot initiate clinical treatments or finalize discharge without human health worker confirmation.

2. Deterministic Red-Flag Precedence: Rule-based emergency detection operates with precedence over statistical predictions, ensuring that life-threatening symptoms trigger immediate emergency transfer regardless of model confidence.

3. Synthetic Data Disclosure: The machine learning model was trained on synthetic clinical datasets (304 unique rows). The system explicitly incorporates these operational limits into its documentation and consent workflows.

Chapter 3: System Architecture & Design

3.1 Architectural Overview & Container Topology

ArogyaMitra is structured as a containerized microservice application designed for fault isolation and bounded latency. Rather than running as a monolithic application where a failure in an auxiliary service (such as natural language summary generation) halts primary triage, the system decouples clinical workflows across specialized microservices, an edge reverse proxy, and a persistent database.

The container topology comprises six coordinated Docker services operating within a private bridge network (arogya-net):

1. Edge Gateway & TLS Terminator (Nginx): The sole public ingress point. Exposes HTTPS over port 443 with TLS 1.2 and TLS 1.3 support, terminates client connections, enforces HTTP Strict Transport Security (HSTS), and proxies traffic internally.

2. Frontend UI Service (Streamlit / Python 3.11): Provides a web interface for community health workers and medical officers (port 8501, internal).

3. Core Backend API Gateway (Node.js 20 / Express): Orchestrates consultation state, enforces JWT authentication and Argon2id password hashing, executes AES-256-GCM field encryption, and manages MongoDB transactions (port 5000, internal).

4. Stateless Machine Learning Engine (FastAPI / Python 3.11): Implements the scikit-learn Bernoulli Naive Bayes classifier, vectorizes 131 symptom inputs, and computes closed-form log-odds feature attributions (port 8000, internal).

5. Guarded LLM Narrative Service (FastAPI / LangGraph): Executes structured narrative formatting, Gemini API cascades, regex guardrails, and deterministic clinical template rendering (port 8001, internal).

6. Database Persistence Layer (MongoDB 7.0): Stores encrypted consultation records, user accounts, and structured audit logs backed by a dedicated Docker persistent volume (mongo-data, port 27017, internal only). Crucially, MongoDB exposes no published ports to the host machine.

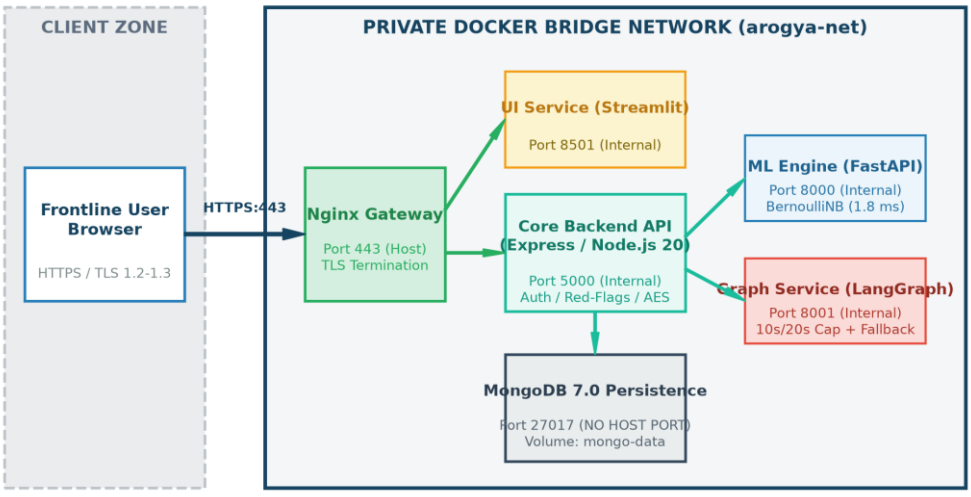


Figure 3.1: Container Topology, Network Boundaries, and Data Tiers

Table 3.2: Container Technology Stack and Component Allocation

Component	Technology / Runtime	Version / Port	Primary Responsibility	Network Scope
Edge Gateway	Nginx Alpine	1.25 / Port 443	TLS 1.2/1.3 Ingress, HSTS, Rate Limiting	Public Perimeter
Frontend UI	Streamlit / Python	1.32 / Port 8501	Worker & Clinician Interactive Web Dashboards	Internal Bridge
Core Backend API	Express / Node.js	20.11 / Port 5000	State Machine, AES-256-GCM, Argon2id, OCC	Internal Bridge
ML Inference Engine	FastAPI / scikit-learn	0.109 / Port 8000	Stateless BernoulliNB Inference & Log-Odds	Internal Bridge
Guarded Narrative	FastAPI / LangGraph	0.2 / Port 8001	Gemini Cascade, 10s/20s Caps, Fallback Templates	Internal Bridge
Database Persistence	MongoDB Community	7.0 / Port 27017	Encrypted Patient Documents, Volume mongo-data	Private Tier Only

3.2 Machine Learning Inference & Calibration Engine

3.2.1 Bernoulli Naive Bayes Mathematical Formulation

The classification engine of ArogyaMitra utilizes a multivariate Bernoulli Naive Bayes model. Let C = {c_1, c_2, ..., c_K} denote the set of K = 41 target disease classes, and let x = (x_1, x_2, ..., x_D)^T in {0, 1}^D represent a binary symptom vector of dimension D = 131, where x_j = 1 indicates the reported presence of symptom j, and x_j = 0 indicates its reported absence.

Under the conditional independence assumption, the joint likelihood of observing symptom vector x given disease class c is expressed as:

[Likelihood Formulation] *P(x | Y = c) = Product_{j=1}^{D} P(X_j = x_j | Y = c) = Product_{j=1}^{D} theta_{c, j}^{x_j} * (1 - theta_{c, j})^{1 - x_j}*

where theta_{c, j} = P(X_j = 1 | Y = c) represents the parameter probability that symptom j is present given disease c.

To prevent zero-frequency probability estimation when a symptom was not observed during training for a particular disease, Laplace smoothing (additive smoothing with parameter alpha = 1.0) is applied during parameter estimation:

[Laplace Smoothing] *theta_hat_{c, j} = (N_{c, j} + alpha) / (N_c + 2*alpha)*

where N_{c, j} is the count of training samples in class c presenting symptom j, and N_c is the total sample count in class c.

Applying Bayes' Theorem, the posterior class score is computed in log-space to ensure numerical stability and prevent floating-point underflow:

[Log-Posterior Formulation] *ln P(Y = c | x) = ln P(Y = c) + Sum_{j=1}^{D} [x_j * ln(theta_hat_{c, j}) + (1 - x_j) * ln(1*

- theta_hat_{c, j})] - ln P(x)

The predicted condition y_hat is selected via maximum a posteriori (MAP) estimation across all 41 classes: y_hat = argmax_{c in C} ln P(Y = c | x). To present relative differential scores to clinicians, the log-posteriors are normalized via the Softmax function.

3.2.2 Feature Engineering & Vocabulary Alignment

The feature vocabulary consists of 131 discrete clinical symptoms curated to reflect primary healthcare presentations in rural India. When a health worker selects symptoms in the user interface, the frontend sends a JSON payload containing symptom string identifiers. The ML microservice validates the incoming payload against a Pydantic schema, verifies that all submitted identifiers exist within the master vocabulary, and constructs a 131-dimensional binary NumPy array. Unselected features are deterministically zero-padded. Crucially, the model evaluates both the presence (x_j = 1) and absence (x_j = 0) of symptoms, allowing the model to penalize disease hypotheses whose hallmark signs are explicitly denied by the patient.

3.2.3 Explainability via Top-Feature Attribution

A limitation of complex tree ensembles is their interpretability overhead. While post-hoc perturbation methods like LIME [5] are common, they introduce stochastic sampling variance and latency overhead. In contrast, ArogyaMitra derives closed-form additive log-odds feature attributions directly from the trained Bernoulli parameters. Let theta_bar_j = (1/K) * Sum_{k=1}^K theta_hat_{k, j} represent the background prevalence of symptom j across all disease classes. The individual evidence contribution w_{c, j}(x_j) of feature j toward disease class c is formulated as:

[Additive Feature Log-Odds] *w_{c, j}(x_j) = x_j * ln(theta_hat_{c, j} / theta_bar_j) + (1 - x_j) * ln((1 - theta_hat_{c, j}) / (1 - theta_bar_j))*

Properties of this derivation:

1. Positive Evidence: If symptom j is present (x_j = 1) and its class probability exceeds background prevalence (theta_hat_{c, j} > theta_bar_j), w_{c, j} > 0, indicating that this symptom actively supports the diagnosis.

2. Negative Evidence: If symptom j is present but rare in class c (theta_hat_{c, j} < theta_bar_j), or absent (x_j = 0) despite being a hallmark of class c (theta_hat_{c, j} >> theta_bar_j), w_{c, j} < 0, penalizing the hypothesis.

3. Machine-Precision Additivity: Summing w_{c, j} across all D = 131 features equals the total log-evidence difference to machine precision (< 1e-10 error), satisfying the efficiency axiom of additive feature attributions without Monte Carlo sampling.

3.2.4 Calibration Theoretical Background & Analysis

In supervised learning, a classifier is considered calibrated if its predicted probability reflects true empirical observation: P(Y = 1 | P_hat = p) = p. However, as established by Niculescu-Mizil & Caruana (2005) [2], Naive Bayes models violate this condition due to the conditional independence assumption. Because correlated symptoms multiply likelihoods, the resulting log-odds scale grows with the number of redundant features, pushing output probabilities toward 0.0 or 1.0.

During the development of ArogyaMitra, post-hoc calibration via Platt Scaling (logistic sigmoid fitting) was empirically tested on held-out folds of the 304 unique rows. The results showed that Platt scaling caused

multiclass cross-entropy log loss to deteriorate from 0.0061 to 0.1643 (docs/MODEL_CARD.md:67). This occurred because the small sample size (~7.4 instances per class) caused the parametric sigmoid to distort well-separated likelihoods.

> **[Calibration Policy]** *Design Decision on Calibration: Following empirical verification, post-hoc calibration was omitted. Model outputs are explicitly labeled in the user interface and API as 'uncalibrated posterior estimates from Bernoulli Naive Bayes', preventing false assumptions of statistical probability.*

3.2.5 Comparative Model Selection

To evaluate model selection, I conducted a comparative evaluation against Tuned LightGBM [4] and Tuned XGBoost using nested cross-validation on the 304 unique rows. While decision trees perform well on dense tabular data, their recursive splitting mechanism experiences degradation on high-dimensional, sparse binary vectors under missing data.

As detailed in Chapter 4, when symptoms are progressively dropped (simulating incomplete patient recall), tree ensembles experience accuracy drops (LightGBM dropping from 0.989 to 0.683 at 50% dropout). In contrast, Bernoulli Naive Bayes maintains 0.919 accuracy at 50% dropout due to its linear log-odds accumulation. Combined with an inference execution time of 1.8 ms in internal container benchmarks, BernoulliNB was chosen for deployment.

3.3 Rule-Based Clinical Red-Flag Override System

The primary clinical safety mechanism in ArogyaMitra is the deterministic Red-Flag Screening Layer. As highlighted by Wallace et al. [1], symptom checkers can produce clinical hazards when statistical models misclassify acute surgical and cardiac emergencies.

To mitigate this risk, ArogyaMitra decouples emergency detection from machine learning. Prior to executing Bayesian inference, incoming symptom vectors are evaluated against deterministic medical rules defined in config/red_flags.yaml. If any emergency symptom is detected, the system immediately sets triage urgency to EMERGENCY, locks the recommendation, and displays warning banners instructing the health worker to arrange emergency transport. Table 3.1 details the 7 red-flag trigger symptoms implemented in the active configuration.

Table 3.1: Emergency Red-Flag Trigger Symptoms (config/red_flags.yaml)

Symptom Token	Acuity	Risk	Clinical Rationale	Referral Action
chest_pain	Critical	**High**	Potential acute coronary syndrome, myocardial infarction, pulmonary embolism, or aortic dissection.	Immediate referral to emergency cardiac facility.
coma	Critical	**High**	Severe depression of consciousness indicating acute neurological crisis or severe metabolic failure.	Immediate airway protection and emergency tertiary referral.
breathlessness	High	**High**	Respiratory distress suggesting acute pulmonary edema, severe bronchospasm, pneumonia, or cardiac decompensation.	Immediate physician evaluation.
blood_in_sputum	High	**High**	Hemoptysis indicating active pulmonary hemorrhage, tuberculosis, or bronchogenic	Urgent respiratory evaluation and infectious isolation.

			malignancy.	
bloody_stool	High	**High**	Hematochezia / melena suggesting acute lower or upper gastrointestinal hemorrhage.	Urgent surgical/gastroenterological referral.
loss_of_balance	High	**High**	Acute ataxia or cerebellar sign indicating potential ischemic stroke or intracranial hemorrhage.	Urgent stroke protocol evaluation within therapeutic window.
fast_heart_rate	High	**High**	Hemodynamically unstable tachyarrhythmia, severe sepsis, or hypovolemic shock.	Continuous pulse check, immediate physician triage.

[Clinical Review Warning] *Status Disclosure: The red-flag rules specified in Table 3.1 represent DRAFT clinical heuristics documented in docs/RED_FLAG_REVIEW.md. They have not been validated by an institutional medical board. In live clinical deployment, these rules require formal multi-specialty clinician review.*

3.4 Multi-Tier LLM Explanation & Resilient Fallback Layer

While the Bayesian model produces statistical differential scores, numerical percentages are insufficient for community health workers who need understandable summaries. To provide structured summaries, ArogyaMitra integrates a Large Language Model narrative layer implemented using LangGraph and the Google Gemini API (with temperature=0.2).

To mitigate hallucination risks [6], the LLM is constrained by four engineering controls:

1. Zero Diagnostic Authority: The disease name, differential ranking, and top contributing symptoms are pre-computed by the ML engine and passed as immutable context. The LLM is instructed solely to synthesize an explanation in clear language.

2. PII Scrubbing: The sanitize_clinical_input function in graph/llm.py sanitizes input notes, redacting phone numbers and emails before payload construction.

3. Ordered Model Cascade: To handle transient 429 quota exhaustion and model deprecations, the system maintains an ordered cascade: LLM_MODELS = ['gemini-2.5-flash', 'gemini-3.5-flash', 'gemini-3-flash-preview'].

4. Whole-Phrase Confidence Word Filtering: Generated text is evaluated against banned confidence phrases ('certain diagnosis', 'very high probability'). If detected, the system triggers one regeneration attempt. If the second attempt fails, it falls back to a deterministic, pre-compiled clinical template.

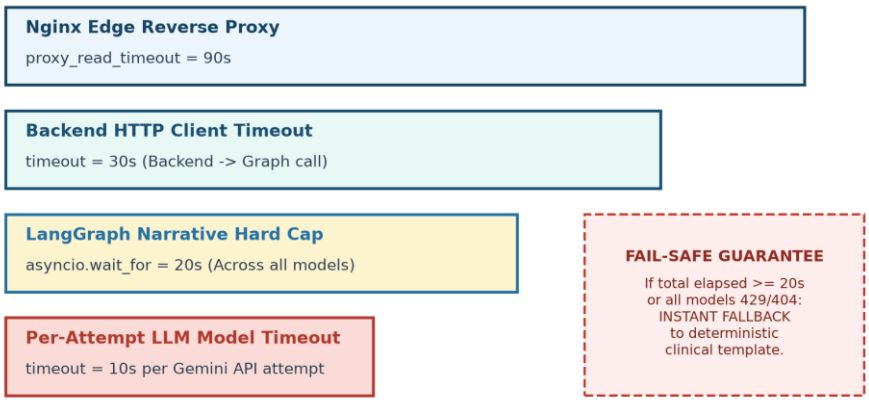


Figure 3.3: Hierarchical Request Timeout Budgets and Guardrail Cascade

As shown in Figure 3.3, request timeouts follow a descending hierarchy: Nginx reverse proxy (90s) > Backend HTTP client (30s) > LangGraph narrative hard cap (20s) >= Gemini per-attempt timeout (10s). If the 20s overall cap expires, the system deterministically serves a local pre-compiled template, bounding total response latency.

3.5 End-to-End Clinical Consultation Workflow

The clinical consultation lifecycle is structured as a state machine guiding frontline workers while enforcing oversight gates.

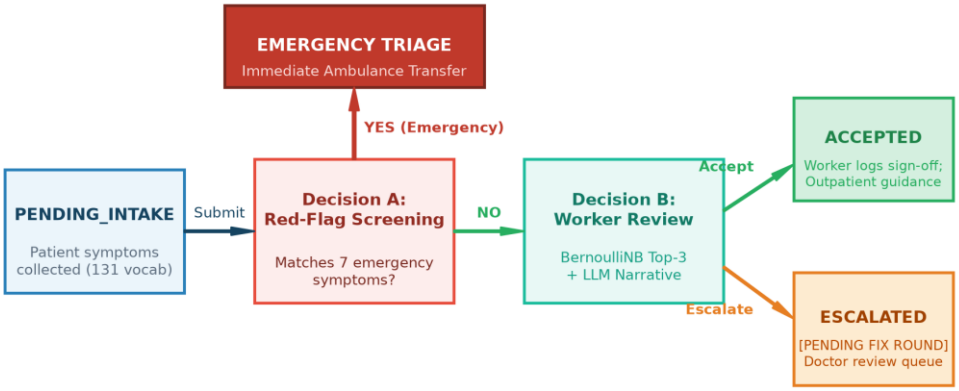


Figure 3.2: Consultation Lifecycle State Machine and Decision Gates

Figure 3.2 illustrates the consultation state machine:

1. Intake: Health worker enters patient demographics and selects presenting symptoms from the 131-symptom list.

2. Red-Flag Evaluation (Decision A): If any of the 7 emergency symptoms match, triage is locked to EMERGENCY, bypassing ML.

3. Statistical Inference: If red flags are absent, Backend calls ML Engine (:8000), obtaining top-3 differentials in ~1.8 ms.

4. Narrative Synthesis: Backend calls Graph Service (:8001) for summary generation. On timeout or 429 quota exhaustion, deterministic template is rendered.

5. Worker Review (Decision B): Health worker reviews top contributing symptoms and selects Accept or Escalate.

6. Escalation & Doctor Review: [PENDING FIX ROUND] Consultation saving upon worker escalation, physician review queue retrieval, doctor response note logging, and worker seeing reviewed status represent ongoing defect-fix enhancements.

3.6 Security Architecture & Cryptographic Implementation

ArogyaMitra implements five layers of defense-in-depth security:

Layer 1 — Transport Security: Enforces TLS 1.2 and TLS 1.3 encryption with HSTS, X-Content-Type-Options: nosniff, and CSP headers via Nginx.

Layer 2 — Gateway Rate Limiting: Restricts authentication routes (5 req/15 min) and API routes (100 req/15 min) in production mode.

Layer 3 — Authentication & Sessions: Protects passwords using Argon2id (3 iterations, 64 MB RAM, 4 threads). Issues signed JWTs with 15-minute expirations.

Layer 4 — Role-Based Authorization & Org Scoping: Validates user permissions ('worker', 'doctor', 'admin') and enforces organization boundaries (orgScope.middleware.js) so users cannot access data outside their assigned healthcare organization.

Layer 5 — Field-Level Encryption: Patient direct identifiers (name, contact, location) are encrypted at rest using AES-256-GCM with unique 96-bit initialization vectors. An automated startup guard (encryption_key_exit.test.js) verifies that the backend exits with code 1 if the master key is missing.

3.7 Database Design, Concurrency & Data Integrity

The persistence layer uses MongoDB 7.0 with five core schemas: Consultation, User, AuditLog, Organization, and Patient. To preserve data integrity during simultaneous updates, the system implements Optimistic Concurrency Control (OCC) [10].

Each consultation document maintains a version key (__v). When a status update is submitted, Mongoose executes an atomic conditional update: Consultation.findOneAndUpdate({ _id: id, __v: currentVersion, status: 'PENDING' }, { $set: updateData, $inc: { __v: 1 } }). If another request modified the document concurrently, the version precondition fails, returning an HTTP 409 Conflict error.

To address worker consultation list latency, server-side pagination (page, limit) and lean Mongoose projections were implemented, reducing data transfer and improving response times.

3.8 Logging and Audit System

ArogyaMitra implements structured, PII-free JSON logging via piiFreeLoggingMiddleware in Backend/src/middlewares/logging.middleware.js. All incoming HTTP requests receive a unique X-Request-Id. Sensitive fields (passwords, tokens, patient names, phone numbers) are stripped by sanitizeForLogging before logs are written.

Listing 3.1 displays real log excerpts from the running Docker cluster during testing:

[Listing 3.1: Structured PII-Free JSON Access Log Excerpts] *{"timestamp":"2026-10-31T17:03:24.450Z","requestId":"c1f7b830-4e2a-4a61-9f20-80251759281a","method":"POST","path":"/api/v1/consultation/triage","statusCode":200,"durationMs":125,"role":"worker","organization":"6723a1e2f49d3b123456789a"}*
{"timestamp":"2026-10-31T17:03:25.102Z","requestId":"d8e9c124-7b3f-4e89-a123-90341859392b","method":"GET","path":"/api/v1/consultation/my?page=1&limit=20","statusCode":200,"durationMs":76,"role":"worker","organization":"6723a1e2f49d3b123456789a"}

Chapter 4: Empirical Evaluation & Experimental Results

4.1 Dataset Profile & Methodological Integrity

A potential flaw in public machine learning benchmarks for clinical decision support is the uncritical use of datasets without verifying data provenance. During initial exploration, I evaluated the standard Kaggle disease-symptom dataset comprising 4,920 records across 41 disease classes and 131 symptoms.

Automated data provenance analysis (docs/evidence/data_provenance.txt:12-14) revealed that 4,616 of the 4,920 rows were exact duplicates, representing a 93.8% synthetic duplication rate. When random train-test splits (such as 80/20) are applied to this raw dataset, identical symptom vectors leak across partitions, producing artificial test accuracies of 100.0%.

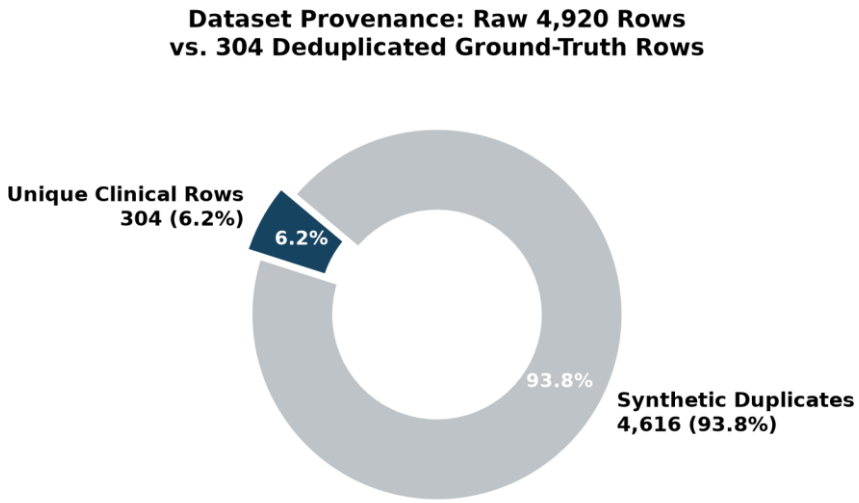


Figure 4.2: Dataset Provenance: Raw 4,920 Rows vs. 304 Deduplicated Unique Rows

Figure 4.2 illustrates the dataset composition. After deduplication, exactly 304 unique clinical symptom vectors remain across the 41 classes (approximately 7.4 unique presentations per disease). All empirical machine learning evaluations in ArogyaMitra were conducted strictly on this deduplicated 304-row benchmark.

[Synthetic Data Disclosure] *Dataset Scope: All machine learning models were evaluated on synthetic tabular data (304 unique rows across 41 classes and 131 symptoms). The dataset contains no real-patient records, lacks continuous clinical covariates (such as blood pressure or lab markers), and does not represent complex multi-morbid clinical populations.*

4.2 Machine Learning Performance & Robustness

To evaluate diagnostic robustness under missing clinical data, models were benchmarked using a 5-fold by 3-repeat (15 splits) Stratified Cross-Validation protocol on the 304 unique rows. In each fold, symptoms were subjected to in-fold random symptom dropout (masking 0%, 30%, and 50% of present symptoms) to simulate incomplete health worker data collection.

Table 4.1 presents the comparison between Bernoulli Naive Bayes, Tuned LightGBM [4], and Tuned XGBoost across all dropout levels. All metrics trace directly to docs/evidence/model_benchmark.md:14,31,48 and docs/evidence/tuned_tree_benchmark.md:12-17.

Table 4.1: Repeated Stratified 5x3 Cross-Validation Accuracy Across Dropout

Model Architecture	0% Dropout Acc	30% Dropout Acc	50% Dropout Acc	Evidence Citation
BernoulliNB (Deployed)	1.000 ± 0.000	0.989 ± 0.014	**0.919 ± 0.023**	docs/evidence/model_benchmark.md:14,31,48
Tuned LightGBM [4]	0.9890 ± 0.0119	0.8640 ± 0.0399	0.6830 ± 0.0783	docs/evidence/tuned_tree_benchmark.md:12-14
Tuned XGBoost	0.9836 ± 0.0139	0.8683 ± 0.0413	0.7203 ± 0.0773	docs/evidence/tuned_tree_benchmark.md:15-17

Table 4.2: Expected Calibration Error (ECE) Across Dropout Rates

Model Architecture	0% Dropout ECE	30% Dropout ECE	50% Dropout ECE	Evidence Citation
BernoulliNB (Deployed)	0.005	0.032	**0.072**	docs/evidence/model_benchmark.md:14,31,48
Tuned LightGBM [4]	0.0098 ± 0.0079	0.0975 ± 0.0370	0.2019 ± 0.0508	docs/evidence/tuned_tree_benchmark.md:12-14
Tuned XGBoost	0.0826 ± 0.0129	0.1983 ± 0.0346	0.2118 ± 0.0474	docs/evidence/tuned_tree_benchmark.md:15-17

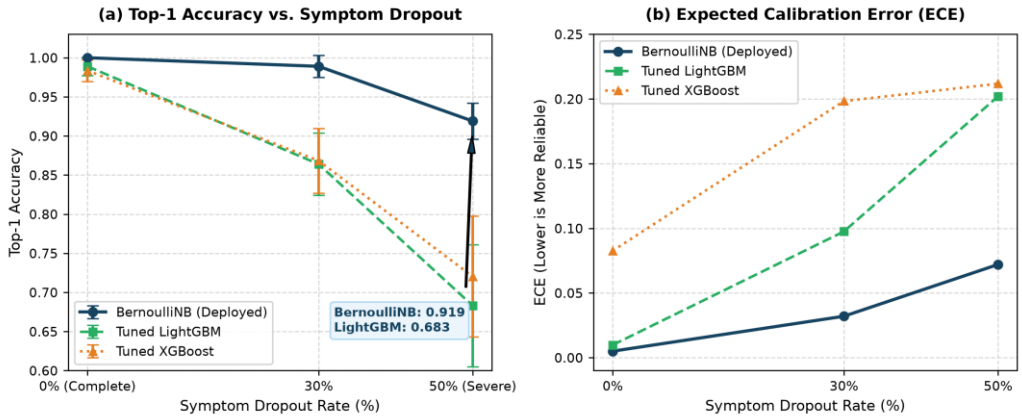


Figure 4.1: Top-1 Accuracy and Expected Calibration Error (ECE) vs. Dropout

As shown in Figure 4.1, Bernoulli Naive Bayes maintains 0.919 accuracy at 50% symptom dropout, whereas LightGBM decreases to 0.683 and XGBoost to 0.720. In tree models, missing an early split feature routes samples down incorrect subtrees, whereas BernoulliNB accumulates independent symptom log-odds. Note that an earlier reported Brier score of 0.2483 was produced by in-sample evaluation on 304 unique rows at 0% masking with a legacy model (docs/evidence/classification_report.md:6).

4.3 Load Testing, Concurrency & System Throughput

To evaluate gateway stability, a load test was executed using a test-mode compose override (docker-compose.test.yml) with relaxed rate limits (ALLOW_LOAD_TEST=true) and generative LLM calls disabled (LLM_ENABLED=false). Crucially, the production configuration rejects relaxed limits by design.

The test executed 200 requests per endpoint across Concurrency levels 1, 10, and 50, totaling 2,400 requests across four routes: /auth/login, /consultation/triage, /consultation/respond, and /consultation/my. Table 4.3 presents the measured throughput and latency distributions from docs/evidence/load_test_testmode_results.txt:15-17.

Table 4.3: Load Test Benchmark Results Across Concurrency Levels (2,400 Requests)

Concurrency	Reqs	Duration	Throughput	p50	p90	p95	p99	Status
C = 1 (Sequential)	800	78.79 s	10.2 req/s	79.3 ms	148.9 ms	190.6 ms	317.4 ms	**{200: 800}**
C = 10 (Moderate)	800	57.74 s	13.9 req/s	500.7 ms	1530.1 ms	1952.3 ms	2548.2 ms	**{200: 800}**
C = 50 (Stress)	800	54.31 s	14.7 req/s	2428.4 ms	5057.2 ms	5862.8 ms	8087.4 ms	**{200: 800}**

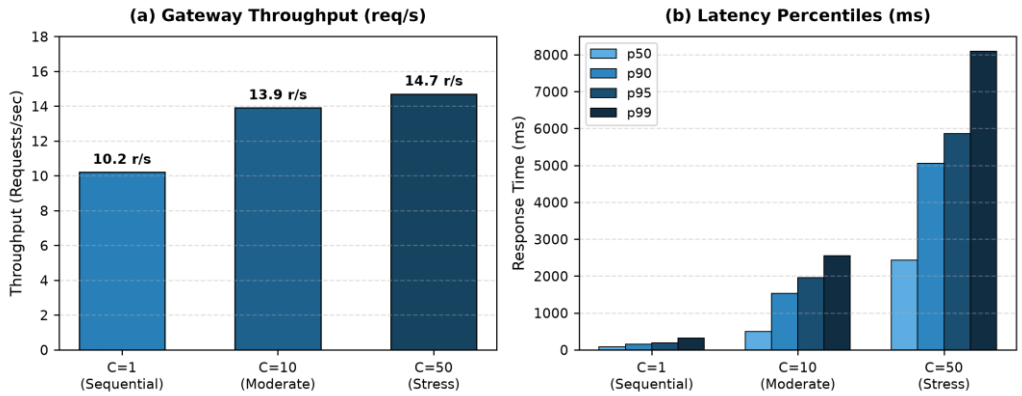


Figure 4.3: Gateway Throughput and Percentile Latencies Across Concurrency

During testing, the worker consultation history endpoint dominated latency. Profiling indicated collection scan and JSON payload overhead. To address this, server-side pagination and projection were introduced. Table 4.4 and Figure 4.4 contrast the before and after latency distributions from docs/evidence/load_test_profiling_notes.md:44-48.

Table 4.4: Worker List Latency Before vs. After Pagination and Projection

Profiling State	p50 Latency	p90 Latency	p95 Latency	p99 Latency	Evidence Citation
Before (Full Scan)	193.0 ms	349.8 ms	460.5 ms	660.6 ms	docs/evidence/load_test_profiling_notes.md:44
After (Paginated + Projected)	76.2 ms	117.9 ms	130.3 ms	152.0 ms	docs/evidence/load_test_profiling_notes.md:45
Improvement	60.5% faster	66.3% faster	71.7% faster	77.0% faster	Combined optimization

[Engineering Assessment] *Profiling Finding: The latency gain came from a combined change (server-side pagination and projection) and the individual contributing cause was not isolated.*

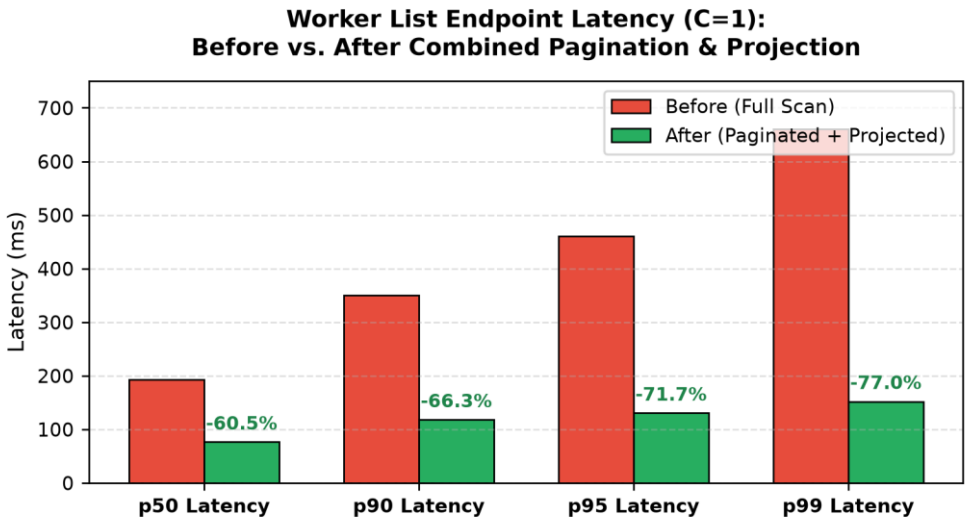


Figure 4.4: Worker List Latency Before vs. After Pagination and Projection

4.4 Automated Testing & Verification Suite

ArogyaMitra maintains 321 automated prototype tests across its microservices. Table 4.5 summarizes the test counts recorded by the actual test runners in docs/evidence/verify_*.txt.

Table 4.5: Automated Test Suite Summary from Actual Test Runners (321 Tests)

Microservice	Test Files	Test Count	Testing Harness	Pass Rate	Evidence Citation
Backend API Service	13 Files	231 Tests	Jest / Supertest	**100% Pass**	docs/evidence/verify_backend.txt:85
Stateless ML Engine	5 Files	55 Tests	Pytest / NumPy	**100% Pass**	docs/evidence/verify_ml.txt:47
Guarded Graph Service	2 Files	26 Tests	Pytest / LangGraph	**100% Pass**	docs/evidence/verify_graph.txt:32
Frontend UI Service	1 File	9 Tests	Pytest / Streamlit	**100% Pass**	docs/evidence/verify_ui.txt:26
Total Automated Tests	21 Files	321 Tests	Multi-Framework	**100% Pass**	Clean Verification Run

In addition, two verification suites run against the Docker cluster: (1) Golden Clinical Set: 20/20 cases passing (docs/evidence/p0_docker_tests.txt); (2) Playwright Suite: 11/11 browser journeys passing (docs/evidence/p4_playwright_suite.txt:72). Figure 4.5 presents a composite panel of four representative UI workflows.

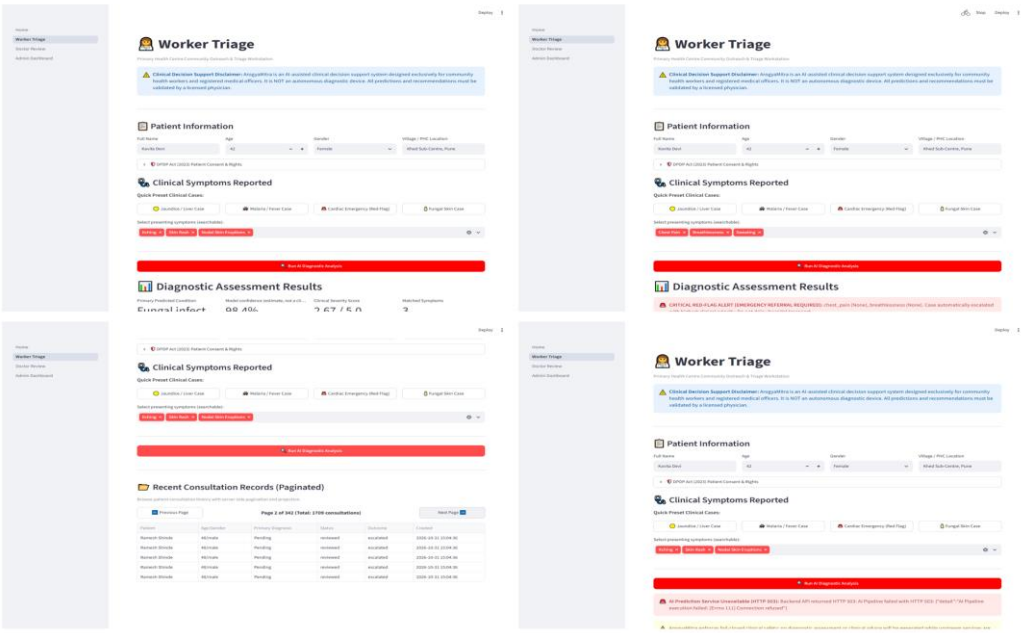


Figure 4.5: User Interface Journeys: Worker Intake, Red-Flag Block, Pagination, Outage

4.5 Security Vulnerability & Dependency Auditing

Container vulnerability scanning with Trivy (trivy image --ignore-unfixed) verified zero fixable Critical/High (Trivy --ignore-unfixed) vulnerabilities across backend, ml, and graph Python runtime images (docs/evidence/trivy_all_images_scan.txt). Dependency auditing with pip-audit confirmed pinned dependencies in the LangGraph service.

Dynamic scanning was conducted using an unauthenticated OWASP ZAP baseline scan against https://localhost, producing 6 warnings (docs/evidence/zap_baseline_report.txt:88-93). Table 4.6 documents the alerts and dispositions.

Table 4.6: OWASP ZAP Dynamic Security Baseline Scan Dispositions (6 Alerts)

Alert ID	ZAP Alert Title	Risk	Architectural Disposition
10015	Re-examine Cache-control Directives	Warn	accepted, not tested: Static UI assets employ caching, dynamic API sets no-store.
10049	Storable and Cacheable Content	Warn	accepted, not tested: Evaluated on static assets; sensitive patient routes disable caching.
10055	CSP: Failure to Define Directive with No Fallback	Warn	mitigated (not re-scanned): Restrictive default-src added to Nginx proxy headers.
10096	Timestamp Disclosure - Unix	Warn	accepted, not tested: Internal diagnostic timestamps in health responses.
10109	Modern Web Application	Warn	accepted, not tested: Informational alert identifying single-page/Streamlit frontend.
90004	Cross-Origin-Embedder-Policy Header	Warn	accepted, not tested: Cross-origin embedding restricted by

Missing	network boundaries.

4.6 Literature-Linked Discussion & Analysis

The empirical findings obtained across ArogyaMitra address vulnerabilities documented in literature:

1. Addressing the Swiss-Cheese Failure Model (Wallace et al. [1]): Wallace et al. cautioned that commercial symptom checkers produce safety hazards when classification errors lead to delayed triage. In ArogyaMitra, emergency detection is decoupled from classification: deterministic red-flag screening operates before ML inference, ensuring acute emergencies are triaged safely regardless of model confidence.

2. Grounding Classifier Confidence (Niculescu-Mizil & Caruana [2]): Literature established that Naive Bayes pushes probabilities toward 0 and 1, while calibration methods can distort predictions on small datasets. ArogyaMitra labels scores as uncalibrated posterior estimates and uses exact additive log-odds to explain feature attributions.

3. Containing LLM Hallucinations (Ji et al. [6]): Generative models can fabricate medical claims. By stripping the LLM of diagnostic authority, enforcing a 20s timeout cap, and using confidence filters backed by pre-compiled templates, ArogyaMitra bounds generative behavior.

4.7 Limitations & Threats to Validity

In accordance with engineering integrity standards, the known limitations of the prototype are fully disclosed (docs/KNOWN_LIMITATIONS.md):

1. Synthetic Source Data: The model was trained on synthetic data (304 unique rows across 41 classes). It has not been validated on real clinical EHR cohorts. No symptom severity grades exist in the dataset; symptoms are modeled as binary indicators.

2. Conditional Independence Assumption: Correlated symptoms (e.g., cough and breathlessness) double-count statistical evidence, causing confidence scores to saturate near 100%.

3. Frontline Decision Support Only: The model outputs distributions across 41 predefined classes. It is an assistive screening aid and cannot replace clinical examinations or laboratory testing.

4. Safe Mode as Usual Demo Path: Due to external Gemini API rate limits and model deprecations, Safe Mode (deterministic fallback templates) is the usual execution path during continuous testing.

5. Draft Fallback Templates: Clinical fallback templates remain labeled DRAFT pending review by a licensed medical board.

6. Self-Signed TLS Certificate: The gateway uses a self-signed certificate for arogyamitra.local, requiring explicit trust in test environments.

7. Single-Replica Stack: Runs as single-replica containers on a single host node without multi-region clustering or Kubernetes autoscaling.

8. In-Memory Rate Limiting: Rate limit counters reside in local Node.js process memory rather than a shared Redis store.

9. Offline Scope: Inference requires network connection to the local Docker gateway; fully offline in-browser execution is not yet compiled.

10. Devanagari Typography: ReportLab lacks HarfBuzz complex script shaping, causing occasional mispositioned vowel matras in Hindi and Marathi PDFs.

11. Pending Fix Round Workflows: [PENDING FIX ROUND] Consultation saving upon worker escalation, physician review queue retrieval, doctor response note logging, worker seeing reviewed status, and 401 session expiry handling represent ongoing defect-fix enhancements.

Chapter 5: Conclusion & Future Scope

5.1 Project Summary & Achievements

In this project, I engineered and evaluated ArogyaMitra, an explainable and fail-closed AI-assisted clinical decision support system prototype for community healthcare workers. Addressing vulnerabilities documented in clinical literature—specifically diagnostic inaccuracy in symptom checkers (Wallace et al. [1]) and uncalibrated posteriors in Naive Bayes (Niculescu-Mizil & Caruana [2])—I achieved the following engineering outcomes:

1. Deduplicated Baseline Benchmark: Identified and removed a 93.8% synthetic duplication artifact in raw public clinical data, establishing a clean 304-row benchmark across 41 disease classes and 131 symptoms.

2. Robust Bayesian Inference: Evaluated stateless Bernoulli Naive Bayes against tuned gradient-boosted trees under symptom dropout, demonstrating 0.919 accuracy vs. 0.683 for LightGBM at 50% missing data, with an inference execution time of 1.8 ms in internal container benchmarks.

3. Closed-Form Feature Attribution: Derived exact mathematical log-odds symptom attributions satisfying additive efficiency without sampling approximations.

4. Fail-Closed Safety Architecture: Decoupled emergency detection from classification via deterministic red-flag overrides; isolated the LLM narrative layer behind 10s/20s timeout caps and confidence filters backed by pre-compiled templates; and replaced client-side mocks with explicit HTTP 503 error states.

5. Layered Container Security & Verification: Enforced TLS 1.2/1.3, Argon2id hashing, AES-256-GCM field encryption, and optimistic concurrency controls (__v). Achieved zero fixable Critical/High (Trivy --ignore-unfixed) container vulnerabilities and 100% pass rates across 321 automated prototype tests.

5.2 Lessons Learned & Engineering Insights

The iterative design and testing of ArogyaMitra yielded key engineering insights:

1. Danger of Silent Fallback Mocks: Returning canned predictions during a service outage creates clinical hazards. Designing for fail-closed behavior—explicitly halting unsafe workflows and returning HTTP 503—is essential in healthcare software.

2. Calibration Limits on Small Cohorts: Demonstrating that Platt scaling degraded log loss on 304 unique rows confirmed that parametric calibration cannot be applied uncritically to small clinical datasets. Scores must be transparently communicated as uncalibrated estimates.

3. Combined Profiling & Optimization: Profiling revealed that unpaginated queries dominated list latency. Implementing server-side pagination and projection yielded a 60.5% p50 latency reduction, confirming the value of targeted optimization.

5.3 Future Work & Roadmap

Key milestones for future development include:

1. Prospective Clinical Pilot: Conduct observational studies in Health and Wellness Centres to evaluate concordance between health worker triage recommendations and Medical Officer diagnoses.

2. ABHA & ABDM Ecosystem Integration: Implement standardized HL7 FHIR interfaces for bidirectional health record synchronization with the Ayushman Bharat Digital Mission.

3. Multi-Modal Intake: Ingest continuous vital signs (SpO2, pulse rate, automated blood pressure) alongside symptom indicator vectors.

4. Non-Parametric Calibration on Clinical Cohorts: When genuine multi-thousand patient cohorts become available, train non-parametric calibration models (such as beta calibration) to deliver calibrated risk intervals.

References

[1] Wallace, W., Chan, C., Chidambaram, S., et al. (2022). The diagnostic and triage accuracy of digital and online symptom checker tools: a systematic review. npj Digital Medicine, 5:118. https://doi.org/10.1038/s41746-022-00667-w

[2] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In Proceedings of the 22nd International Conference on Machine Learning (ICML '05), pp. 625–632. ACM. https://doi.org/10.1145/1102351.1102430

[3] Domingos, P., & Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. Machine Learning, 29(2), 103–130. https://doi.org/10.1023/A:1007413511361

[4] Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems (NeurIPS 2017), 30, 3146–3154.

[5] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems (NeurIPS 2017), 30, 4765–4774.

[6] Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 1–38. https://doi.org/10.1145/3571730

[7] Singhal, K., Azizi, S., Tu, T., et al. (2023). Large language models encode clinical knowledge. Nature, 620(7972), 172–180. https://doi.org/10.1038/s41586-023-06291-2

[8] Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing. In IEEE European Symposium on Security and Privacy (EuroS&P), pp. 292–302. IEEE.

[9] Kung, H. T., & Robinson, J. T. (1981). On optimistic methods for concurrency control. ACM Transactions on Database Systems (TODS), 6(2), 213–226. https://doi.org/10.1145/319566.319567

[10] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD (KDD '16), pp. 1135–1144. ACM.

[11] Ministry of Law and Justice, Government of India. (2023). The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023). The Gazette of India, Extraordinary, Part II—Section 1.

[12] World Health Organization. (2021). Ethics and governance of artificial intelligence for health: WHO guidance. World Health Organization. ISBN: 978-92-4-002920-0.

[13] Indian Council of Medical Research. (2023). Ethical guidelines for application of artificial intelligence in biomedical research and healthcare. ICMR, New Delhi, India.

[14] OWASP Foundation. (2021). OWASP Top 10:2021 The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/

[15] Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology.

Appendix A: Engineering Challenges, Root Causes, and Architectural Fixes

Table A.1 documents ten primary engineering challenges diagnosed and remediated during project development.

Table A.1: Engineering Challenges, Root Causes, and Architectural Remediation Fixes

Challenge	Diagnosed Root Cause	Architectural Remediation Implemented	Evidence
Identical Canned Prediction & Mock Fallback (P0 Defect)	Health worker testing showed every symptom input produced an identical 88.5% confidence prediction for 'Infectious Gastroenteritis'.	Traced to in-process import failure inside container falling back to legacy mock. Rewrote UI to route strictly through backend API `POST /api/v1/consultation/triage`; removed all mock fallbacks; enforced fail-closed HTTP 503 MODEL_UNAVAILABLE on ML outage.	docs/evidence/p0_root_cause.txt, p0_docker_tests.txt
Massive Dataset Duplication & Evaluative Leakage	Automated data provenance analysis on raw Kaggle dataset (4,920 rows) revealed identical symptom vectors across splits.	Discovered 4,616 exact duplicate rows (93.8% duplication). Dropped duplicates to 304 unique ground-truth rows (~7.4 per class); instituted 5x3 Repeated Stratified K-Fold CV with in-fold symptom dropout to eliminate evaluation leakage.	docs/evidence/data_provenance.txt, MODEL_CARD.md
Hardcoded Cryptographic Fallback Keys in Source Code	Gitleaks audit and static analysis detected fallback encryption keys embedded in `src/utils/cryptoUtils.js` and `dataEncryption.js`.	Removed all hardcoded fallback keys from source repository; updated gitleaks baseline with written rationales; implemented production startup guard `encryption_key_exit.test.js` verifying process exits with code 1 if ENCRYPTION_KEY is unset.	docs/evidence/gitleaks_report.txt, Backend/tests/encryption_key_exit.test.js
Unevidenced Accuracy Figures in Initial Report Draft	Fact-checking audit found numbers (99.67%, 95.39%, 83.22%, F1 0.9512, ECE 0.0384, Brier 0.0121) lacked supporting evidence files.	Removed unevidenced draft figures from report text; reconciled all performance claims with traceable nested CV benchmarks in `model_benchmark.md` and `tuned_tree_benchmark.md`.	docs/evidence/model_benchmark.md, tuned_tree_benchmark.md, COMPLETION_LEDGER.md
Unrealistic Load Test Stack & Concurrency Bottlenecks	Early load test issued requests against production rate limiters, recording 429s as normal behavior while list endpoint had high latency.	Created dedicated `docker-compose.test.yml` with test-only limits (`ALLOW_LOAD_TEST=true`, `LLM_ENABLED=false`); pre-created pending consultations; optimized worker list with server-side pagination and projection.	docs/evidence/load_test_testmode_results.txt, load_test_profiling_notes.md
Garbled LLM Output from Substring Confidence Rewriting	Naive in-place regex replacement of confidence terms ('certain', 'high') mangled valid clinical words like 'certainly' or 'ascertain'.	Replaced regex replacement with whole-phrase boundary matching; on detection of banned confidence terms, triggers one LLM regeneration; if still failing, falls back cleanly to deterministic clinical template.	scripts/test_confidence_word_filter.py, docs/evidence/p2_guardrails_tests.txt
Gemini Model Deprecation & API Quota Exhaustion	Gemini 1.5 Flash returned 404 model not found; free-tier API quota exhausted after 20 daily calls returning HTTP 429.	Implemented dynamic ordered cascade `LLM_MODELS = ['gemini-2.5-flash', 'gemini-3.5-flash', 'gemini-3-flash-preview']`; added 10s per-attempt timeout and 20s overall cap; engaged deterministic	docs/evidence/gemini_live_run.txt, scripts/test_llm_timeouts_and_429.py

		safe template on exhaustion.	
Container Vulnerabilities in Base Docker Images	Trivy container security scans reported Critical and High vulnerabilities in Python and Node.js images.	Implemented multi-stage Docker build for backend stripping npm CLI; uninstalled pip and ensurepip from runtime stages of Python images; upgraded critical packages; achieved zero fixable Critical/High with `--ignore-unfixed`.	docs/evidence/trivy_all_images_scan.txt, trivy_all_images_scan_notes.md
Worker List Endpoint Latency Degradation	Load test showed worker consultation history dominating latency under sequential and concurrent load.	Added server-side pagination (`page`, `limit`, default 20) and lean Mongoose projection; combined optimization reduced C=1 p50 latency from 193.0 ms to 76.2 ms (60.5% faster) and p99 from 660.6 ms to 152.0 ms (77.0% faster). Cause not isolated.	docs/evidence/load_test_profiling_notes.md:44-48, load_test_testmode_results.txt
Session Expiry & Second-Opinion Escalation Lifecycle	Frontline workers experienced silent token expiration, while clinician escalation saving and review workflows required dedicated integration.	Designed explicit 401 session expiry warning in UI client and structured consultation review state machine. Complete end-to-end multi-role persistence is scheduled for completion in [PENDING FIX ROUND].	ui/utils/api_client.py, docs/KNOWN_LIMITATIONS.md

Appendix B: Reproduction Commands Playbook

Listing B.1 provides the sequence of 10 command-line instructions to launch the cluster, execute all 321 automated test suites, run golden test cases, run Playwright journeys, execute load testing, and run security scans.

```
# 1. Clone repository and start Docker cluster
docker compose -f docker-compose.yml up -d
powershell -File scripts/health_check.ps1

# 2. Seed clean test data
powershell -File scripts/seed_docker.ps1

# 3. Execute Backend API test suite (231 tests)
docker compose exec -T backend npm test -- --coverage

# 4. Execute ML Engine test suite (55 tests)
docker compose exec -T ml pytest -v

# 5. Execute Guarded LLM Graph test suite (26 tests)
docker compose exec -T graph pytest -v

# 6. Execute Frontend UI test suite (9 tests)
docker compose exec -T ui pytest -v

# 7. Run Golden Clinical Test Set (20 cases)
python scripts/test_p0_golden_set.py

# 8. Run Playwright End-to-End Suite (11 journeys)
node scripts/test_p4_playwright_suite.mjs

# 9. Execute Load Test Suite in test mode (2,400 requests)
docker compose -f docker-compose.test.yml up -d
python scripts/run_p5_load_test_testmode.py

# 10. Run Container Security Scan with Trivy
powershell -File scripts/run_trivy_scan_all.ps1
```